Companion Web Platform Boundary

Diablo-Inspired MMORPG Project

Document Path: /docs/03-technical-design/companion-web-platform-boundary.md
Version: 0.3
Status: Draft
Priority: High
Required By: Pre-Alpha / Alpha Web Platform Planning
Owner: Web Platform Team / Platform Services / Security / Production
Last Updated: TBD

---

1. Purpose

The Companion Web Platform is the non-realtime web application layer that supports the MMORPG project.

It may include account management, public website pages, website content, storefront features, player support tools, admin dashboards, documentation pages, marketing pages, public project updates, and future player-facing services.

The Companion Web Platform should be treated as a related but separate system from the core game runtime.

The core MMO runtime is responsible for real-time gameplay.

The Companion Web Platform is responsible for account, content, commerce, administrative, community, support, and public-facing workflows that do not need to run inside the real-time game simulation.

---

2. Boundary Summary

The Companion Web Platform should not be required to run the moment-to-moment game loop.

The real-time MMO stack should remain focused on:

Godot Client
-> WebRTC / Protobuf
-> Gateway Service
-> C++ Zone Servers

-> C++ Dungeon Instance Servers
-> Game Platform Services
-> PostgreSQL / Redis / NATS

The Companion Web Platform should focus on:

Web Client
-> Web Backend
-> Auth Provider
-> Account Service
-> Content System
-> Storefront / Account APIs
-> Website Database
-> Admin / Support Tools

The two systems may share identity, account status, selected account metadata, entitlements, and operational data through service boundaries, but they should not be tightly coupled in a way that makes the website a dependency for real-time gameplay.

---

3. Core Rule

The web platform supports the game.

It does not simulate the game.

The web platform may display game-related information, manage accounts, support purchases, expose player-facing dashboards, publish news, or support administrative workflows, but it should not own real-time combat, movement, loot generation, dungeon state, monster AI, or authoritative gameplay state.

---

4. Current Web Platform Decisions

The following decisions are currently accepted or tentatively accepted for this phase of planning.

4.1 Web Features Before or During Alpha

The major web platform features should be ready sometime during or before Alpha.

This does not mean every web feature must be final or production-polished. It means the project should have working MVP versions of the major web platform areas before Alpha is complete.

Target web areas include:

- Public website
- Account portal
- Basic profile/account view
- Website content system
- Support entry point
- Basic admin tools
- Storefront / entitlement pathway if commerce is enabled
- Deployment pipeline
- Testing foundation

4.2 Authentication

Auth0 is a candidate long-term identity provider for both website login and game login.

This could simplify authentication by giving both the website and game login flows a common identity provider. However, the benefits and drawbacks still need further review before this is treated as a final long-term decision.

Potential benefits:

- Shared identity provider
- Faster implementation
- OAuth 2.0 / OpenID Connect support
- Built-in login and account security features
- Reduced need to build custom authentication from scratch

Potential concerns:

- Vendor dependency
- Cost at scale
- Migration complexity if replaced later
- Need to cleanly separate web sessions from game sessions
- Need to avoid blindly trusting web login state inside the game runtime

Current planning position:

Auth0 may be used for both website and game login,
but the game runtime must still create and validate its own game session.

Recommended model:

Player authenticates through Auth0
-> Web Platform or Account Service validates identity
-> Game session is created
-> Gateway validates game session
-> Zone server accepts only validated player connection

The game should not blindly trust the presence of a web login alone.

4.3 Account Service Boundary

The web platform should call an Account Service rather than directly owning all account/game identity behavior.

The systems should remain separated for security and practicality.

Reasons:

- The website and game runtime have different security risks.
- The website should not directly mutate game-critical state.
- Account logic can be centralized behind service contracts.
- The game Gateway can validate game sessions independently.
- Future admin, support, and entitlement workflows can use the Account Service without direct database access.

Recommended flow:

Website
-> Web Backend
-> Account Service
-> Account / Identity Records
-> Game Session / Entitlement Validation

4.4 Website Backend Repository Boundary

The website backend should have its own repository available, if not immediately, then soon.

Current planning position:

The website backend should be separated from the game platform services.

Reasons:

- The website platform and game platform have different release cycles.
- The website can move faster without blocking game runtime work.
- The game server stack should not become tangled with website-specific code.
- Web deployment can target Cloud Run while game servers remain Docker/local or staged.
- Web testing, frontend/backend changes, Directus integration, and storefront work can evolve independently.

Recommended repository direction:

website-frontend
website-backend

game-platform-services
game-server-runtime

The exact repository names can be decided later.

4.5 Website Database

The website platform should have its own database system.

The web platform database should be separate from the game runtime database for security, maintainability, and practical development boundaries.

Website-owned data may include:

- CMS content
- Website pages
- Support tickets
- Web account preferences
- Storefront metadata
- Order history references
- Admin dashboard metadata
- Public profile settings
- Web audit records
- Web session metadata

Game-owned data should remain controlled by game services.

Game-owned data includes:

- Characters
- Character stats
- Skills
- Inventory
- Equipment
- Items
- Currency
- Loot
- Dungeon progress
- Combat state
- Zone state
- Game mutation audit logs

The website may request or display selected game data through APIs or read models, but it should not directly mutate game-critical tables.

4.6 Directus Usage

Directus may manage both marketing/content pages and developer-facing documentation.

This is not yet a hard requirement, but it is allowed as a planning direction.

Possible Directus-managed content:

- News posts
- Patch notes
- Public pages
- FAQ pages
- Media posts
- Development blogs
- Support articles
- Public documentation pages
- Developer-facing project docs, if practical

Boundary warning:

Directus should not directly control authoritative gameplay data, live balance data, loot tables, item stats, or server configuration unless a separate reviewed configuration pipeline is created.

4.7 Entitlement Boundary

Stripe entitlements may need to exist in both the web-account layer and the game-account layer.

Recommended interpretation:

- The web platform records orders, purchase history, and storefront-facing entitlement records.
- The game/account service validates and applies game-relevant entitlements.
- The game runtime applies only validated game entitlements.

Recommended flow:

Player purchases product on website
-> Stripe confirms transaction
-> Web backend records order
-> Web entitlement record is created
-> Account / Entitlement Service validates entitlement
-> Game account entitlement is created or updated
-> Game service applies permitted benefit

An entitlement is not automatically an in-game item.

Game-impacting benefits should not be granted directly by the website frontend.

4.8 Admin Tool Location

For now, admin tools may live inside the website platform.

This is acceptable for simplicity during early development if admin routes are strongly protected, role-based, and audited.

Potential benefits:

- Faster development
- Fewer separate apps to maintain
- Shared web authentication
- Shared deployment pipeline
- Easier internal access during early Alpha

Potential drawbacks:

- Larger website attack surface
- Public website and internal tools may become too coupled
- Permission mistakes could become more dangerous
- Admin workflows may eventually need stricter isolation
- Security review becomes more important

Recommended rule:

Early Phase:
Admin tools may live inside the website platform.

Later Phase:
Move admin tools into a separate internal operations app if security, scale, or team workflow requires it.

4.9 Public Website Timing

The public website should launch before the playable Alpha.

The public website does not need to be content-heavy at first.

Minimum public website scope:

- Landing page
- Project overview
- Development update area
- Media/mock-up area
- Contact/community link
- Basic account portal link if available
- Support or feedback entry point
- Alpha information page when appropriate

4.10 Website Deployment

The website should deploy through Cloud Run while the game servers remain Docker/local or staged until Kubernetes and Agones are needed.

Recommended model:

Website Platform:
Google Cloud Run
Docker
GitHub Actions
Managed database
Directus
Auth0
Stripe if commerce is enabled

Game Runtime:
Docker first
Local/staging services first
Kubernetes foundation
Agones foundation
Full Kubernetes/Agones later when needed

This allows the website to become available early without forcing the real-time game infrastructure to be production-ready too soon.

---

5. Companion Web Platform Responsibilities

The Companion Web Platform may own or support the following systems.

5.1 Public Website

Responsibilities:

- Landing page
- Game overview
- News posts
- Development updates
- Media gallery
- Screenshots
- Trailer embeds
- Patch notes
- Community links

- Documentation links
- Press or portfolio material
- Alpha information page when appropriate

Primary teams:

- Web Team
- Creative / Content Team
- Production

5.2 Account Portal

Responsibilities:

- Login and account access
- Account profile
- Email update
- Password/auth provider management
- Linked account display
- Security settings
- Account status
- Support links
- Future account recovery workflows

Primary teams:

- Web Team
- Platform Services
- Security

5.3 Player Profile Portal

Potential responsibilities:

- Character list display
- Character summary
- Class display
- Level/progression summary
- Public/private profile settings
- Achievement display
- Future guild display
- Future match or dungeon history

Important boundary:

The website may read selected character data for display, but it should not directly mutate real-time gameplay state.

Primary teams:

- Web Team
- Platform Services
- Database
- Security

5.4 Storefront

Potential responsibilities:

- Game purchase page
- Cosmetic product pages
- Future DLC or expansion pages
- Payment checkout flow
- Order history
- Refund/support links
- Entitlement display

Important boundary:

The storefront may create purchase records or web entitlement records, but the game backend must validate and apply any game-impacting entitlement through controlled service contracts.

Primary teams:

- Web Team
- Platform Services
- Security
- Production

5.5 Content Management

Responsibilities:

- News articles
- Patch notes
- Static pages
- Media posts
- Development blogs
- FAQ pages
- Support articles
- Public documentation pages
- Developer-facing documentation if approved

A headless CMS such as Directus may be used for website content management.

Important boundary:

CMS-managed content should not directly control authoritative gameplay balance unless a separate reviewed configuration pipeline is created.

Primary teams:

- Web Team
- Content Team
- Production

5.6 Admin Dashboard

Potential responsibilities:

- Account lookup
- Character lookup
- Support review
- Ban/suspension tools
- Player reports
- Server status display
- Deployment status links
- Operational dashboards
- Audit log search
- Customer support workflows

Important boundary:

Admin tools must use strict access control and audit logging.

Primary teams:

- Web Team
- Platform Services
- Security
- Operations
- Support

5.7 Support Portal

Potential responsibilities:

- Help articles
- Bug report form
- Player support tickets
- Ban appeal process
- Account recovery assistance
- Known issue tracker
- Troubleshooting guides

Primary teams:

- Web Team
- Support
- QA
- Security

5.8 Documentation / Portfolio Site

Potential responsibilities:

- Public technical case study
- Architecture summaries
- Project milestone pages
- Media kit
- Development roadmap excerpts
- Portfolio-facing engineering summaries
- Recruit/collaborator information

Primary teams:

- Production
- Documentation
- Web Team

---

6. Companion Web Platform Non-Responsibilities

The Companion Web Platform should not own:

- Real-time combat simulation
- Movement validation
- Monster AI
- Zone state
- Dungeon encounter state
- Loot generation
- Item creation
- Item duplication checks
- Direct inventory mutation
- Currency mutation
- XP rewards
- Server tick loop
- AoI calculations
- Snapshot replication

- WebRTC gameplay transport
- C++ zone server lifecycle during early Alpha

Some of these systems may be displayed or requested through web tools, but the authoritative action must remain owned by the correct game service.

---

7. Recommended Web Platform Stack Boundary

The Companion Web Platform may use a separate stack from the core game runtime.

Recommended companion stack:

Frontend:
- Angular
- Angular SSR
- Playwright
- Jest
- Angular Testing Utilities

Backend:
- Node.js
- NestJS
- Fastify Adapter
- Fastify Inject
- Jest
- Testcontainers

Database / ORM:
- Dedicated website database
- PostgreSQL or Neon PostgreSQL
- Drizzle ORM

Content:
- Directus

Authentication:
- Auth0
- OAuth 2.0
- OpenID Connect

Payments:
- Stripe

Hosting / Deployment:
- Google Cloud Run
- Docker
- GitHub Actions

This stack should be documented in the dedicated Website Platform Architecture documents rather than fully embedded inside the MMO Technical Design Document.

The MMO Technical Design Document should only describe the boundary and integration points.

---

8. Core MMO Stack vs Companion Web Stack

Area	Core MMO Runtime	Companion Web Platform
Primary purpose	Real-time gameplay	Account, website, content, support, storefront
Client	Godot	Angular
Realtime transport	WebRTC	HTTPS / API calls
Serialization	Protobuf	JSON / REST / GraphQL if selected
Simulation	C++ zone servers	None
Backend services	TypeScript platform services	NestJS + Fastify Adapter
Account access	Game session validation	Auth0 candidate + Account Service
Source of truth	Game PostgreSQL database	Dedicated website database
Cache	Redis	Redis optional
Messaging	NATS / NATS JetStream	Optional event integration
Deployment	Docker first, Kubernetes/Agones later	Cloud Run / Docker / CI/CD
Auth	Game session validation	Auth0 / OAuth / OIDC candidate
Testing	C++, TypeScript, protocol, integration	Jest, Playwright, Fastify Inject, Testcontainers

9. Integration Points

The Companion Web Platform may integrate with the game stack through carefully controlled service boundaries.

Potential integration points:

Auth0
-> Account Service
-> Game Account Records

Website Account Portal
-> Player Profile API
-> Character Summary Read Model

Storefront
-> Stripe
-> Web Order Record
-> Entitlement Service
-> Game Account Entitlements

Admin Dashboard
-> Support/Admin API
-> Audit Logs / Account Status

Directus
-> Public Website
-> News / Patch Notes / Static Content / Documentation

The web platform should prefer service APIs over direct database mutation when interacting with game-owned state.

---

10. Identity and Authentication Boundary

The web platform may use Auth0 with OAuth 2.0 and OpenID Connect.

Auth0 is currently a candidate for both website login and game login because it may simplify identity management. This decision still needs review for benefits, drawbacks, cost, vendor lock-in, and long-term maintainability.

The same identity provider may support both website login and game login, but the game runtime still needs its own validated game session.

Recommended model:

Player authenticates through Auth0 or selected identity provider
-> Account Service validates identity
-> Game session is created
-> Gateway validates game session
-> Zone server accepts only validated player connection

The website may handle account login, but the Gateway and game services must still validate that a player is allowed to enter the world.

---

11. Data Ownership Boundary

Data ownership should be clearly separated.

11.1 Game-Owned Data

Game-owned data includes:

- Characters
- Character stats
- Skills
- Inventory
- Equipment
- Items
- Currency
- Loot
- Dungeon progress
- Combat state
- Zone state
- Audit logs for game mutations

This data should be modified only by game-authoritative services.

11.2 Web-Owned Data

Web-owned data may include:

- Website pages
- News posts
- CMS entries
- Marketing content
- Store product pages
- Support tickets
- Public account preferences
- Web session metadata
- Non-authoritative display data
- Web audit records
- Web admin dashboard metadata

11.3 Shared or Referenced Data

Shared or referenced data may include:

- Account identity
- Entitlements
- Public profile settings
- Display name
- Account status
- Support flags
- Ban/suspension status

Shared data must have a clearly defined source of truth.

---

12. Database Boundary

The web platform should use its own database system.

The game platform should use its own game database.

The two systems should communicate through controlled APIs and services.

Recommended early rule:

- Keep game-critical tables isolated from direct web writes.
- Allow the web platform to read safe summary data through APIs or read models.
- Route sensitive updates through platform services.
- Use audit logging for admin actions and entitlement changes.
- Use the Account Service for cross-boundary account operations.

Direct website writes to game-critical tables should be avoided unless explicitly approved by the Technical Lead and Security Lead.

---

13. Entitlement Boundary

If the website or storefront sells access, cosmetics, founder packs, expansions, or other account-level products, those purchases should create entitlements.

An entitlement is not the same as an in-game item.

Recommended flow:

Player purchases product on website
-> Stripe confirms transaction

-> Web backend records order
-> Web entitlement record is created
-> Account / Entitlement Service validates entitlement
-> Game account entitlement is created or updated
-> Game service applies permitted benefit

Game-impacting rewards should not be granted directly by the website frontend.

---

14. Admin Tool Boundary

Admin tools may initially live inside the website platform for simplicity.

However, they must be treated as sensitive internal tools.

Admin actions should:

- Require authentication
- Require authorization
- Use role-based permissions
- Emit audit logs
- Avoid direct database mutation where possible
- Use reviewed service endpoints
- Support rollback or investigation when practical

Examples of admin-sensitive actions:

- Ban account
- Modify account status
- Grant entitlement
- Inspect inventory
- Remove duplicated item
- Review audit logs
- Reset stuck character
- Resolve support ticket

Future consideration:

If the public website and admin tooling become too tightly coupled, the admin dashboard should be moved into a separate internal operations app.

---

15. Website Deployment Boundary

The Companion Web Platform can use a different deployment model from the real-time game stack.

The web platform may be deployed through:

- Docker
- GitHub Actions
- Google Cloud Run
- Managed PostgreSQL / Neon PostgreSQL
- Directus
- CDN or static hosting for public assets

The game runtime may later use:

- Docker
- Kubernetes
- Agones
- Dedicated game server nodes
- STUN/TURN infrastructure
- NATS
- Redis
- PostgreSQL
- Observability stack

This separation allows the website to move quickly without forcing the real-time game infrastructure to be production-ready too early.

---

16. Testing Boundary

The Companion Web Platform should have its own testing strategy.

Recommended web tests:

- Frontend unit tests
- Angular component tests
- Playwright end-to-end tests
- Backend unit tests
- Fastify Inject request tests
- Testcontainers integration tests
- Auth flow tests
- Payment webhook tests
- CMS content tests

- Admin permission tests

The MMO runtime should keep its own technical testing strategy for:

- WebRTC
- Protobuf
- Movement
- Combat
- Inventory
- Zone simulation
- Dungeon instances
- Persistence
- Load testing
- Exploit testing

---

17. Security Boundary

The website increases the project's attack surface.

Security concerns include:

- Account takeover
- Payment abuse
- Admin dashboard abuse
- Permission mistakes
- Exposed support tools
- CMS compromise
- Token leakage
- Insecure webhooks
- Unvalidated entitlement grants
- Unsafe direct database access

Security requirements:

- Use HTTPS for all web traffic.
- Validate all auth tokens server-side.
- Protect admin routes.
- Audit sensitive admin actions.
- Verify payment webhooks.
- Do not trust frontend purchase state.
- Keep secrets out of source control.
- Separate public CMS roles from admin/system roles.
- Avoid direct mutation of game-critical tables from the website.

- Use the Account Service for cross-boundary identity and account operations.

---

18. Local Development Boundary

The web platform may use its own local Docker Compose setup.

A local web development environment may include:

Angular frontend
NestJS/Fastify backend
Dedicated website PostgreSQL database
Directus
Test database
Optional Redis

The game local development environment may include:

Godot client
C++ zone server
TypeScript game services
Game PostgreSQL database
PgBouncer
Redis
NATS
STUN/TURN test setup if needed

These environments may eventually be combined for full-stack integration testing, but they should remain understandable as separate layers.

---

19. Documentation Boundary

The Companion Web Platform should have dedicated documents.

Recommended web platform documents:

website-platform-architecture.md
frontend-architecture.md
backend-architecture.md
website-database-schema-overview.md
account-security.md
website-deployment-runbook.md
website-testing-strategy.md

website-implementation-backlog.md
website-architecture-decision-records.md

The MMO Technical Design Document should reference these documents instead of duplicating all web-specific implementation detail.

---

20. Recommended Rule for This Project

The MMO game runtime and Companion Web Platform should be developed as related but separately documented systems.

Recommended boundary:

Core MMO Runtime:
Owns real-time gameplay, persistence, simulation, inventory, combat, zones, dungeons, and game state.

Companion Web Platform:
Owns public website, account portal, website content, storefront, support tools, admin dashboards, player-facing web views, and public project presentation.

Account Service:
Acts as the controlled boundary between website identity/account workflows and game account/session workflows.

Shared systems should be introduced through explicit APIs, clear ownership, and Architecture Decision Records.

---

21. Resolved and Tentative Questions

The following questions from the previous draft are now aligned with current project answers.

21.1 Should the web platform and game platform share one account database, or should the web platform call an Account Service?

Decision:

The web platform should call an Account Service.

Reason:

The systems should remain separated for security and practicality.

Status:

Accepted for current planning.

21.2 Should Auth0 be the long-term identity provider for both website and game login?

Decision:

Auth0 could be used for both website and game login for simplicity, but the benefits and cons need further review.

Status:

Tentative / needs evaluation.

Notes:

The team should compare:

- Auth0 simplicity
- Cost
- Vendor lock-in
- Security benefits
- Integration complexity
- Migration difficulty
- How game sessions are created after web identity validation

21.3 Should the website backend be a separate repository from the game platform services?

Decision:

Yes. The website backend should have a separate repository available, if not immediately, then soon.

Status:

Accepted for current planning.

Notes:

The website platform should be separated from game platform services to reduce coupling and allow independent development/deployment.

21.4 Should the website use its own PostgreSQL database or a shared database with strict schema separation?

Decision:

The website should have its own database system.

Status:

Accepted for current planning.

21.5 Should Directus manage only marketing/content pages, or also developer-facing documentation?

Decision:

Possibly both.

Status:

Tentative.

Notes:

Directus may be useful for marketing pages, public content, documentation pages, and possibly developer-facing documentation if it remains practical.

21.6 Should Stripe entitlements be game-account entitlements, web-account entitlements, or both?

Decision:

Both.

Status:

Accepted for current planning.

Notes:

The web platform should track storefront/order entitlement state, while the game/account layer should validate and apply game-relevant entitlements.

21.7 Should admin tools live inside the website platform or in a separate internal operations app?

Decision:

For now, admin tools may live inside the website platform.

Status:

Accepted for early planning.

Notes:

This should be revisited later if security, complexity, or workflow separation requires a dedicated internal operations app.

21.8 Which web features are required before Alpha, if any?

Decision:

All major web platform features should be ready sometime during or before Alpha.

Status:

Accepted for current planning.

Notes:

These may be MVP versions, not final production-polished systems.

21.9 Should the public website launch before the playable Alpha?

Decision:

Yes.

Status:

Accepted.

21.10 Should website deployment use Cloud Run while game servers remain Docker/local until Kubernetes and Agones are needed?

Decision:

Yes.

Status:

Accepted.

---

22. Remaining Open Questions

The following questions remain open for later review:

1. What is the minimum public website feature set before Alpha?
2. What account features must exist before Alpha?
3. Which admin actions are allowed inside the website platform?
4. Should admin tools eventually move into a separate internal operations app?
5. Should Directus manage internal developer documentation or only public-facing documentation?
6. What is the first storefront scope, if any?
7. What entitlement types will exist before Alpha?
8. Should web profile pages expose character information before Alpha?

9. How should website data and game account data be synchronized for public profiles?
10. What is the exact local Docker Compose structure for the website platform?
11. What are the benefits and drawbacks of Auth0 for long-term game login?
12. What repository structure should be used for the website frontend, website backend, and game services?

---

23. Related Documents

- website-platform-architecture.md
- frontend-architecture.md
- backend-architecture.md
- website-database-schema-overview.md
- account-security.md
- website-testing-strategy.md
- website-implementation-backlog.md
- architecture-overview.md
- gateway-service-design.md
- account-schema.md
- security-overview.md
- admin-access-control.md
- deployment-runbook.md
- github-actions-runbook.md
- architecture-decision-records.md

---

24. Change Log

Version 0.3

Corrected resolved question mapping.

Updated decisions:

- Web platform should call an Account Service.
- Auth0 for both website and game login is tentative and needs benefits/cons review.
- Website backend should have its own repository available soon.
- Website should have its own database system.
- Directus may manage both marketing/content pages and developer-facing documentation.
- Stripe entitlements may exist in both web-account and game-account layers.
- Admin tools may live inside the website platform for now.

- Major web features should be ready sometime during or before Alpha.
- Public website should launch before playable Alpha.
- Website deployment should use Cloud Run while game servers remain Docker/local until Kubernetes and Agones are needed.

Version 0.2

Updated with resolved planning decisions, later corrected in Version 0.3.

Version 0.1

Initial expanded Companion Web Platform Boundary created.